

Prompting Fundamentals on Real Tasks v2

In this assignment, it's all about getting to understand how to properly prompt in a refined and detailed manner. The assignment wants us to use the tasks in our old assignments, but I feel as though those tasks are too open ended to really improve the prompting, so I came up with a different idea.

Okay, I didn't really come up with another idea, I just liked the API capstone for the Backend track and wanted to make a more specific one for me and maybe even learn React while I'm at it.

First Prompt

“create a web based grocery list maker for me.”

It's a very simple, to the point prompt, but from what I've learned till now, it's too vague and lacks attention to detail. I didn't give it anything to work with besides what I want it to be, I won't be satisfied with the results and I really am not. Even though I could've come up with a better version, it's bad.

Results from Prompt

A grocery list should feel less like a generic to-do app and more like an actual shopping list — grouped by aisle, easy to check off, and it should remember what you added. Let me build that.

Created a file, read a file

Created a file, read a file

Here's the quick tour of what's under the hood, since you're building your own dev chops:

- **Auto-categorization:** type "milk" or "2 chicken breasts" and it gets sorted into an aisle (Produce, Dairy, Meat, Pantry, etc.) via keyword matching against the item name. It's a naive substring match, not anything fancy — good enough for groceries, and a decent first pass if you ever want to swap it for something smarter.
- **Persistence:** it uses the artifact's window.storage API instead of localStorage (which doesn't work in Claude's artifact sandbox) — so your list survives a page refresh or closing the tab, scoped privately to you.

- **No framework:** plain HTML/CSS/vanilla JS, one file. Worth opening the file and skimming — it's a clean example of the render-from-state pattern (mutate items[], call render(), done) if you haven't built something that way before.

The screenshot shows a web application titled "AISLE". At the top right, it says "1 left to grab" and "3 in the cart". Below the title is a search bar containing the text "milk, 2 apples, paper towels...". To the right of the search bar is a quantity input field with the number "1" and an "ADD" button. Below the search bar, there are two sections: "4 DAIRY & EGGS" and "9 OTHER". Each section contains a list of items with checkboxes. In the "DAIRY & EGGS" section, "peanut butter" is unchecked, while "milk" and "eggs" are checked. In the "OTHER" section, "fedbull" is checked. Each item has a red "X" button to its right. At the bottom right, there is a "Clear checked" button.

Notes on results

One thing worth noting for your portfolio: this categorization approach (keyword → bucket) is basically a toy rules engine. If you want a slightly meatier project to talk about in interviews, you could extend this into "smarter" categorization — e.g. fuzzy matching for typos, or letting users teach it new keywords — and that's a much better story than "I made a to-do list."

Iteration 1: Role Assignment

This was hard for me because the roles I was thinking of adding were creeping into the context end instead of what it's actually meant to be. I used Claude to guide me through role assignment since the resources weren't too helpful to me in a way, and the main direction I wanted to go was into getting to think about the app in a more globalized way, make it easier to be an actual app with good features, and we ended up on a developer who works in the supermarket sector, it was too vague because it could be an inventory system designer or TPS system designer, and we went to focusing on shopping for customers. **This was the final prompt I decided on:**

“You are a developer specializing in customer-facing retail and shopping apps. Create a web based grocery list maker for me”

Results from Prompt

Built it as a receipt — makes sense for grocery: category "aisle" sections, a running item tally, and a stamp effect when you check things off.

Quick rundown of what's in it:

- Add items with quantity + category (Produce, Dairy, Bakery, etc.) — they auto-group under aisle headers
- Tap the checkbox to mark got-it (strikethrough + red stamp)
- Remaining-items tally updates live
- "Clear checked" vs "Clear all" as separate actions, since those are pretty different operations
- Editable title, keyboard support (Enter to add, focus states for accessibility)

One thing worth flagging since you're building portfolio pieces: state lives in JS variables only, nothing persists on refresh. That's intentional — browser storage isn't reliable in this sandbox — but for a real portfolio piece you'd want to wire this to localStorage (trivial swap) or a backend if you want it multi-device. Happy to walk through either if you want to extend it.

The screenshot shows a web application titled "GROCERY LIST" with a date "SUNDAY, AUGUST 2, 2026". At the top, there is a form to add items with a text input "add an item...", a quantity input "1", a category dropdown "Other", and an "ADD" button. Below this, the list is organized into sections: "DAIRY & EGGS" and "OTHER". Under "DAIRY & EGGS", there is an unchecked "Egg" with a quantity of "x6" and a checked "Cheese" with a quantity of "x1" and a red "GOT IT" stamp. Under "OTHER", there is a checked "Redbull" with a quantity of "x1" and a red "GOT IT" stamp. At the bottom, there is a tally "ITEMS REMAINING 1" and two buttons: "CLEAR CHECKED" and "CLEAR ALL".

Notes on results

It leaned more into the gimmick than the actual code. It's still a very good version, it just removed the automatic category assignment part and made it manual, not much was done or at least anything that's not visual. Weirdly enough, I believe the first iteration is better than this one. On functionality, that is.

Iteration 2: Context and Motivation

This was hard because the line between motivation and adding features is a very thin line to cross. It's something I can already tell that I will still do a few days later not because I don't understand it or force of habit, it'll be because it's a subconscious thing you just do. This time I didn't argue that much with Claude because it explained the harm of adding a why into motivation, which is still weird to me because the whole iteration is context and motivation but I do get the fact that adding functions or other ideas to this iteration would be cheating in a way. It will be a hard thing to do every time though. **This is the prompt I used:**

"You are a developer specializing in customer-facing retail and shopping apps. Create a web based grocery list maker for me. I want it to help shoppers who forget to re-add something that finished a while ago or buying too much of something they have a lot of"

Results from Prompt

I'll check the frontend design skill's guidance is applied — building this as a distinctive HTML artifact rather than a generic form. Let me build it now.

Created a file, read a file

Created a file, read a file

Here's **Restock Radar** — the two problems you named map to two distinct mechanisms, not just UI copy:

"Forgot to re-add it" — every time you check an item off as bought, its timestamp goes into that item's purchase history. Once an item has 2+ purchases, the app computes the average gap between them. If today's gap since last purchase is $\geq 85\%$ of that average, it surfaces in the **Restock Radar** strip up top as a shelf-tag card. Before there's enough

history, it falls back to sane per-category defaults (produce ~6 days, pantry ~28 days, etc.) so it's not useless on day one.

"Buying too much" — the inverse check runs when you *add* an item: if you're re-adding something you bought way sooner than its usual interval (under half the expected gap), it flags it inline as a soft warning rather than blocking you — you're often right that you're out, the app just doesn't know that.

A few implementation notes worth knowing about, since you're building a portfolio and this pattern will come up again:

- **Persistence:** uses the artifact window.storage API (personal, not shared) instead of localStorage, which doesn't work in Claude artifacts. Worth remembering if you build other artifacts.
- **Backfill form:** real usage takes weeks to generate meaningful history, so I added a collapsed "log past purchase" form that synthesizes two purchase timestamps from "repeats every X days" + "last bought Y days ago" — lets you demo the radar working immediately instead of staring at an empty state.
- **The over-buy detection is naive on purpose** — it's just "gap since last purchase vs. historical average," no inventory tracking of actual quantity on hand. A more rigorous version would need you to log how much you *used*, not just how much you bought, which is a much heavier ask of the user. Worth thinking about if you extend this — there's a real tradeoff between accuracy and friction here that's worth articulating in a portfolio writeup.

Try adding "Milk" a couple times with different gaps via the backfill form to see both signals fire.

RESTOCK RADAR

A grocery list that notices what you keep forgetting — and what you keep over-buying.

▲ RESTOCK RADAR

Things you usually buy again by now, but aren't on your list.

Nothing due for a restock yet. Mark items as bought on your list to start building this up.

ADD TO LIST

ITEM

e.g. Milk

QTYUNITCATEGORYAdd item

1e.g. cartonBeverages

⚠️ You already bought Redbull today — usually lasts about 6 days. Adding anyway is fine, just flagging it.

▶️ Already have purchase history for something? Log it here so Radar can use it right away.

YOUR LIST

Check items off as you buy them — that's how Radar learns your rhythm.

● PRODUCE

☐ Bread × 1

⚠️ possibly already stocked

Have some? ×

● BEVERAGES

☐ Redbull × 1

⚠️ possibly already stocked

Have some? ×

Notes on results

It's definitely closer to what i imagined when I first thought of a Grocery List. It added a tracker which remembers what you checked off the list the first time and the second time you buy it, it warns you. It is good and cool, but I think it becomes a problem when you have a very repetitive list, will it now flag it every month or second week? I understand it's not supposed to suddenly make the whole thing as top notch as Takealot or something, but I feel like every iteration slowly moves it into the generic, industry/market ready stuff; it's interesting. Of course, it makes sense because that's what the role I gave it would create, and the context will push it that way.

By adding the problem and why, it worked to fix the context and motivation I gave it and stopped creating a main idea app, but an app focused on the context and motivation. It sounds like I'm going in circles but the more context, the smaller the circle becomes of what the app is.

Iteration 3: Few-shot examples

I had a hard time on this one because it was hard to pinpoint cases that would need some training for the AI to create the app. It's a new concept so it will be hard, but I don't know man. I just feel like it's better as a part of debugging, but I also understand that the whole concept for this is to have one prompt that will deliver with a fully checked list. Definitely interesting. I heavily relied on my AI assistant to help me come up with few-shot examples,

I don't think I have it in me to really think of such, at least for this example. I tend to struggle when facing open-ended tasks. **But this is the prompt I used:**

“You are a developer specializing in customer-facing retail and shopping apps. Create a web based grocery list maker for me. I want it to help shoppers who forget to re-add something that finished a while ago or buying too much of something they have a lot of.

Use these examples to judge repurchase patterns:

Milk: normal cadence every 5-7 days. Bought again after 6 days → not overbuying, matches normal pace.

Toilet paper: normal cadence every 21 days. Bought again after 20 days → not overbuying, essentially normal pace.

Coffee: normal cadence every 14 days. Bought again after 6 days → overbuying, less than half the normal gap.

Salt: normal cadence every 90 days. Bought again after 25 days → overbuying, far under normal pace for a slow-use item.

Cereal: normal cadence every 10 days. Bought again after 7 days → borderline — a bit sooner than usual, but not a clear overbuy. Use judgment rather than flagging automatically.

Paper towels: normal cadence every 18 days. Bought again after 5 days → overbuying, well under normal pace.”

Results from Prompt

Here's the logic worth understanding, since you're trying to actually learn this stuff, not just get a file:

The core detector is a ratio, not a diff. For any two consecutive purchases of the same item, $\text{ratio} = \text{actual_gap} / \text{normal_cadence}$. Three bands:

- $\text{ratio} \geq 0.8 \rightarrow$ on pace (milk $6/6=1.0$, TP $20/21=0.95$)
- $0.5 \leq \text{ratio} < 0.8 \rightarrow$ borderline, shown but not auto-flagged (cereal $7/10=0.7$ — this is the "use judgment" case, so it gets a neutral amber badge instead of a red one)
- $\text{ratio} < 0.5 \rightarrow$ overbuying (coffee $6/14=0.43$, salt $25/90=0.28$, paper towels $5/18=0.28$)

I checked all six of your examples against this and they land exactly where you specified. The 0.8/0.5 cutoffs aren't arbitrary — they're the tightest thresholds that separate your six

examples correctly, so if you add more test cases later and something misclassifies, that's your signal to adjust the boundaries, not a bug.

The "forgot to re-add" side uses the same ratio but in the other direction: for items *not* currently on the list, $\text{ratio} = \text{days_since_last_purchase} / \text{cadence}$. Above 1.15 it surfaces as a suggestion, above 1.5 it's flagged as "likely out." I seeded a Dish Soap example (32 days since purchase, 20-day cadence, ratio 1.6) so you can see that panel populate immediately.

Cadence itself isn't static. New items get a cadence from a small built-in library (the six you gave me, plus common groceries) or default to 14 days if unknown. Once an item has 3+ logged purchases and you haven't manually overridden its cadence, it switches to `cadenceSource: 'learned'` and recalculates as the rolling average of gaps — so the model adapts to the actual shopper instead of a hardcoded number.

Data persists per-browser via the storage API (not shared/public), so it survives refreshes but isn't just sitting in `localStorage` getting wiped on cache clear.

One thing to think about for your portfolio writeup: the learned-cadence logic silently overwrites a "default" cadence once there's enough data, but it never reconciles with the *classification history* that was computed under the old cadence — so a badge you saw last week could've been computed against a different cadence than what's stored now. Worth deciding if that's a feature (recency-weighted) or a bug (inconsistent audit trail) for your resume story.

RESTOCK · REPURCHASE PATTERN TRACKER

Never re-buy salt you already have.

Log what you actually buy and Restock learns each item's normal rhythm — then flags when you're stocking up too soon, and reminds you when something's probably run out.

Add an item

Known items (milk, coffee, salt...) auto-fill a typical cadence. New items start with no pattern until you log a repurchase.

Item name

Category

Pantry

Normal cadence (days)

auto

Add to list

Shopping list

1 item

OTHER

☐ **Redbull** Overbuying

bought after 6d

usual: 10d

» Purchase history & pattern log

Might be worth re-adding

Not on your list, but well past its usual pace based on when you last bought it.

Dish Soap

32d since last buy · usual gap 28d · likely out

Add

Notes on results

The system is much better now; it went back to the more normal look without any gimmick, even the generic, market ready look is gone which I love. It seems like the idea of adding a restock watcher removed it from the generic look. It added a graph that even shows and explains why the item was flagged and when it's in the early-but-fine period of the month and the ready-to-buy period of the month. It even has an area where it reminds you of something you haven't added in a while which I didn't even think of but is a great idea. I know there's a lot I could be noting instead of these things, but I feel like they're the notable changes in the result.

The changes moved from just being cosmetic to full on integration of new features which i find to be cool. Didn't change much besides just adding rules about how to act from the motivation and it found a way around the rules while still abiding to them.

The ratios and math is a bit concerning though because it has limited scopes which could cause problems not even later but immediately, which is concerning.

Iteration 4: Output Structure

I don't really have an option to talk about the output that's not file related. I believe just stating that I want it to split the app into HTML, CSS and add a README.md doesn't change anything or add a function to the app. So, this is the prompt I used:

“You are a developer specializing in customer-facing retail and shopping apps. Create a web based grocery list maker for me. I want it to help shoppers who forget to re-add something that finished a while ago or buying too much of something they have a lot of. Use these examples to judge repurchase patterns: Milk: normal cadence every 5-7 days. Bought again after 6 days → not overbuying, matches normal pace. Toilet paper: normal cadence every 21 days. Bought again after 20 days → not overbuying, essentially normal pace. Coffee: normal cadence every 14 days. Bought again after 6 days → overbuying, less than half the normal gap. Salt: normal cadence every 90 days. Bought again after 25 days → overbuying, far under normal pace for a slow-use item. Cereal: normal cadence every 10 days. Bought again after 7 days → borderline — a bit sooner than usual, but not a clear overbuy. Use judgment rather than flagging automatically. Paper towels: normal cadence every 18 days. Bought again after 5 days → overbuying, well under normal pace. Provide it in different files for the HTML, CSS and a README.md”

Results from Prompt

The app changed again, it provided a place where you can change whether you buy an item every month, week, or whichever days after. It kept the tracker there to show you when to buy that grocery again. It keeps simplifying the input area after every prompt. Besides that, it just divided the web page into different files.

EST. TODAY · NO. 0001

LEDGER

a grocery list that remembers your pace

Add an item...

every 14

Add

Careful – you bought Redbull only 0d ago. Usual pace is every 14d.

– YOUR LIST –

Redbull

every ~14d · bought 0d ago, usually longer

OVERBUYING

Bought

x

– PANTRY PACE –

Cereal

last bought 7d ago · usual pace every 10d

015d

usual cadence

ON PACE

Coffee

last bought 6d ago · usual pace every 14d

021d

usual cadence

ON PACE

Milk

last bought 6d ago · usual pace every 6d

09d

usual cadence

ON PACE

Paper Towels

last bought 5d ago · usual pace every 18d

027d

usual cadence

ON PACE

Redbull

last bought 0d ago · usual pace every 14d

FIRST BUY

Notes on results

It surprised me when it gave back a JavaScript file because every other iteration didn't have JavaScript; the Readme file is thorough, and its naming convention is clearer than you'd think.

Iteration 5: Step Decomposition

This is where you control the AI's thinking process, instead of just working everything out in bulk, you tell it the steps to take. For me, this was hard because I couldn't really breakdown the steps, of course I can think, it has to work on the form and processing of the data, work on the UX of interacting with the form then you have to work on the look and then you display it, but with those steps, It would've missed a lot, so with AI assistance I got a better understanding of everything. **This is the prompt I came up with:**

“You are a developer specializing in customer-facing retail and shopping apps. Create a web based grocery list maker for me. I want it to help shoppers who forget to re-add something that finished a while ago or buying too much of something they have a lot of. Use these examples to judge repurchase patterns: Milk: normal cadence every 5-7 days. Bought again after 6 days → not overbuying, matches normal pace. Toilet paper: normal cadence every 21 days. Bought again after 20 days → not overbuying, essentially normal pace. Coffee: normal cadence every 14 days. Bought again after 6 days → overbuying, less than half the normal gap. Salt: normal cadence every 90 days. Bought again after 25 days → overbuying, far under normal pace for a slow-use item. Cereal: normal cadence every 10 days. Bought again after 7 days → borderline — a bit sooner than usual, but not a clear overbuy. Use judgment rather than flagging automatically. Paper towels: normal cadence every 18 days. Bought again after 5 days → overbuying, well under normal pace. Provide it in different files for the HTML, CSS and a README.md. Build this in the following order: data model first, then work on classification logic, after that work on the UI/UX then work on the documentation”

Results from Prompt

Done — four files, all in /mnt/user-data/outputs/pantry-rhythm/: index.html, style.css, app.js, README.md. Drop them in one folder and open index.html — no build step, no server, persists to localStorage.

Quick note on the one deviation from your file list: you asked for HTML/CSS/README, but a working app needs logic to live somewhere, so I put the data model + classification logic + UI wiring in app.js rather than inlining it in the HTML — kept it separate so you can read it as its own thing, in the exact build order you specified (model → classification → wiring, all commented as three numbered sections in the file).

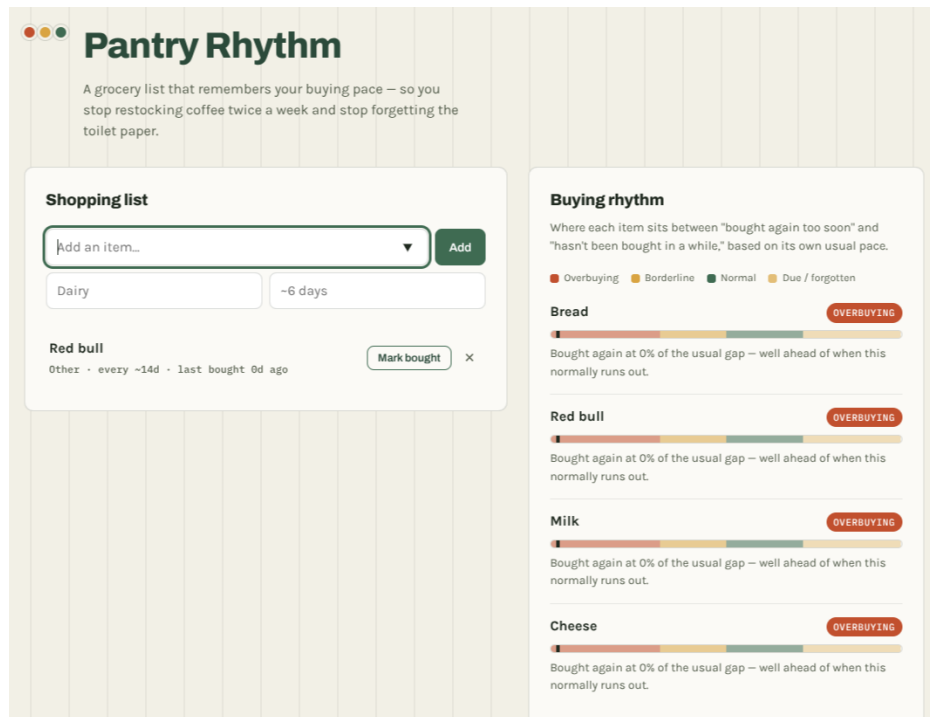
The classification core is a ratio, not a fixed day count:

$$\text{ratio} = \text{daysSinceLastPurchase} / \text{cadenceDays}$$

<0.5 overbuy, 0.5–0.8 borderline, 0.8–1.15 normal, 1.15–1.6 due soon, >1.6 probably forgotten. I ran your six examples through it and they all land where you said they should — table's in the README if you want to double check my arithmetic instead of trusting me.

One thing worth poking at critically: the 0.8–1.15 "normal" band width and the 1.6x cutoff for "forgotten" are both judgment calls I made to fit your six data points — you only gave me

two normal examples (0.86 and 0.95 ratio) and no examples at all for the "due soon" or "forgotten" ends, so those two zones are extrapolated, not verified against a worked example. If you've got more real data, that's the part I'd stress-test first.



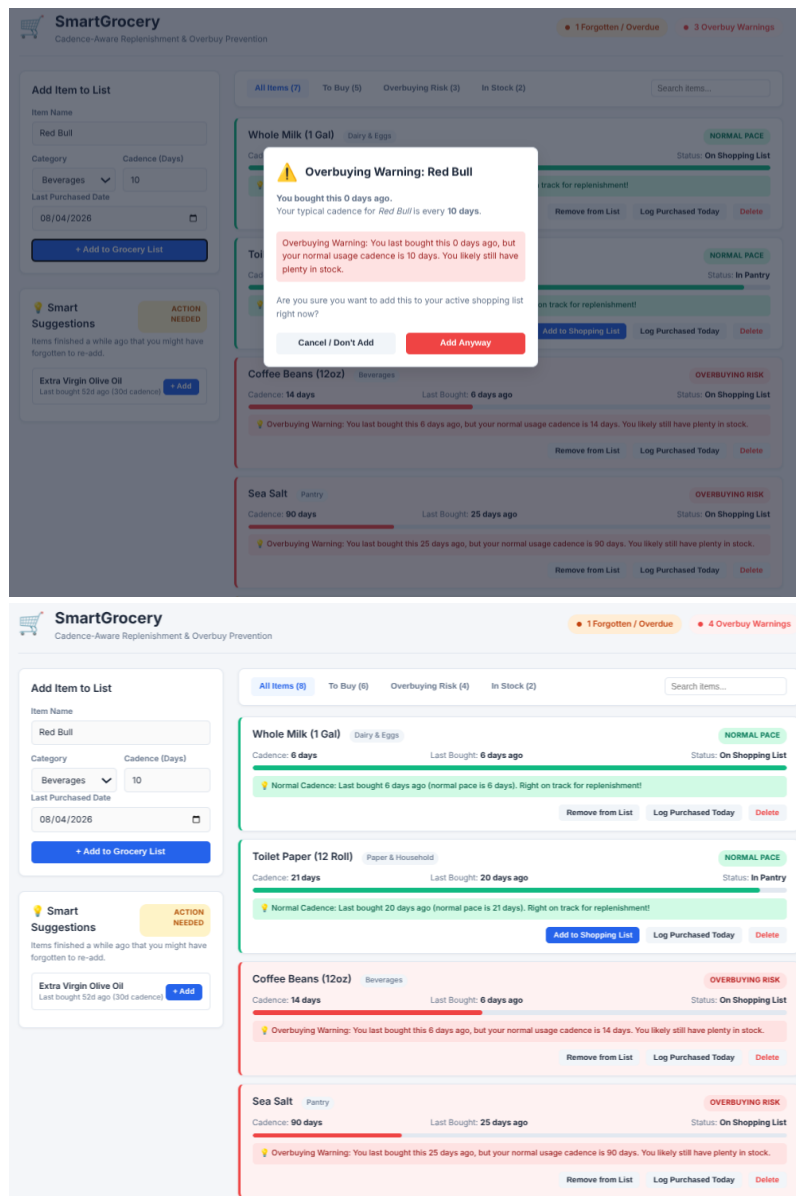
Notes on results

I can't see much change from the first and last besides the UI, of course there's new logic and JavaScript, but from the last iteration to now, it didn't do much. I believe the UI is much more mature, very modern while still having that "wholesale" feel, close to something that would be for Apple or something, very Mac-like. In the logic, it even called out the mistake that was in Iteration 3 and fixed it which is good. It shows it through the logic instead of speeding through it.

Gemini Version

Gemini didn't really explain the results, it just jumped into the README.md which it messed up because it displayed the results and not the actual README, but besides that everything is good. Even the programming is very very similar, but the most different is the

UI. It went more into I'd say Dashboard than app in my opinion. It focused more on the tracking of groceries than on the app actually being a grocery list. Even the logic is a bit different.



“2. Classification Logic Engine (CadenceClassifier)

Rather than applying rigid binary cuts, the classification logic computes a **Cadence Elapsed Ratio**:

$$\text{Cadence Ratio} = \frac{\text{Days Elapsed Since Last Purchase}}{\text{Normal Cadence Days}}$$

I personally like the claude version but the Gemini one is still impressive, and yes, the prompt still works similar on a different engine.